

Function Exercises - Answers

Preparation (Test Data)

We will use the following DNA sequences as example data.

```
dna1 <- c("A", "G", "C", "T", "G")
dna2 <- c("A", "T", "A", "T", "A", "T")
dna3 <- c("G", "G", "C", "C", "G", "G")
dna4 <- c("N", "N", "A", "T", "G")
dna5 <- c("A", "G", "X", "T")  # unknown character test

dna_list <- list(dna1, dna2, dna3, dna4)
```

Level 1 — Easy

Exercise 1: `square_value(x)`

Expected output: `square_value(4) = 16`

```
square_value <- function(x) {
  # x: number given to the function

  result <- x * x
  # result: the square of x (x multiplied by x)

  return(result)
  # return(): specifies the value that the function sends back
}

square_value(4)
```

```
## [1] 16
```

What did we learn?

- A function receives input (`x`)
 - It performs a calculation
 - It returns the result using `return()`
-

Exercise 2: `dna_length(dna)`

Expected output: `dna_length(dna1) = 5`

```
dna_length <- function(dna) {
  # dna: a vector containing bases like A, T, G, C

  length_value <- length(dna)
  # length(): returns how many elements are in the vector

  return(length_value)
}

dna_length(dna1)
```

```
## [1] 5
```

What did we learn?

- `length()` returns the size (number of elements) of a vector.
-

Level 2 — Medium (for + if + counter)

Exercise 3: `gc_count(dna)`

Expected output: `gc_count(dna1) = 3`

```
gc_count <- function(dna) {

  GC_total <- 0
  # counter: starts at 0 and increases when we find G or C

  for (n in dna) {
    # for loop: goes through each base in the DNA sequence
    # n: the current base (for example "A" or "G")

    if (n == "G" || n == "C") {
      # if condition: checks whether the base is G or C
      # || means "OR"

      GC_total <- GC_total + 1
      # if G or C is found, increase the counter by 1
    }
  }

  return(GC_total)
  # return the total number of G and C bases
}

gc_count(dna1)
```

```
## [1] 3
```

What did we learn?

- Counter logic: start with 0, increase with +1 when the condition is true
 - Using a `for` loop to iterate over elements
 - Using `if` to check conditions
-

Exercise 4: `gc_ratio(dna)`

Expected output: `gc_ratio(c("G","C","A","T")) = 0.5`

```
gc_ratio <- function(dna) {  
  GC_total <- 0  
  
  for (n in dna) {  
    if (n == "G" || n == "C") {  
      GC_total <- GC_total + 1  
    }  
  }  
  
  ratio <- GC_total / length(dna)  
  # GC ratio = (number of G + C) / (total number of bases)  
  
  return(ratio)  
}  
  
gc_ratio(c("G","C","A","T"))
```

```
## [1] 0.5
```

What did we learn?

- To calculate a ratio we use: **part / total**
-

Level 3 — Medium / Advanced (Classification)

Exercise 5: `gc_status(dna)`

Expected output: `gc_status(dna3) = "GC-rich"`

```
gc_status <- function(dna) {  
  
  ratio <- gc_ratio(dna)  
  # instead of writing the loop again we call the gc_ratio() function
```

```

if (ratio > 0.5) {
  # if ratio is greater than 0.5, GC content is high
  return("GC-rich")
} else {
  return("GC-poor")
}
}

gc_status(dna3)

```

```
## [1] "GC-rich"
```

What did we learn?

- Calling another function inside a function keeps the code shorter and cleaner.

Level 4 — Hard (Input checking + QC)

Exercise 6: dna_check(dna)

Expected output

```
dna_check(dna1) = "OK"
```

```
dna_check(dna5) = "ERROR: Unknown base"
```

```

dna_check <- function(dna) {

  error_exists <- FALSE
  # logical variable (boolean)
  # FALSE = no error, TRUE = error exists

  for (n in dna) {

    if (n != "A" && n != "T" && n != "G" && n != "C" && n != "N") {
      # if n is not A AND not T AND not G AND not C AND not N
      # then it means there is an invalid character
      # && means "AND"

      error_exists <- TRUE
    }
  }

  if (error_exists == TRUE) {
    return("ERROR: Unknown base")
  } else {
    return("OK")
  }
}

dna_check(dna5)

```

```
## [1] "ERROR: Unknown base"
```

```
dna_check(dna1)
```

```
## [1] "OK"
```

What did we learn?

- Data validation: checking if the input is correct
 - Using TRUE / FALSE for logical checks
-

Exercise 7: dna_quality(dna)

Expected output: dna_quality(dna4) = "Uncertain DNA"

```
dna_quality <- function(dna) {  
  
  N_exists <- FALSE  
  # if N appears we consider the DNA uncertain  
  
  GC_total <- 0  
  # GC counter  
  
  for (n in dna) {  
  
    if (n == "N") {  
      N_exists <- TRUE  
      # if we see N anywhere -> TRUE  
    }  
  
    if (n == "G" || n == "C") {  
      GC_total <- GC_total + 1  
    }  
  }  
  
  if (N_exists == TRUE) {  
    # first check N  
    return("Uncertain DNA")  
  } else {  
  
    ratio <- GC_total / length(dna)  
  
    if (ratio >= 0.6) {  
      return("High GC")  
    } else {  
      return("Normal DNA")  
    }  
  }  
}
```

```
dna_quality(dna4)
```

```
## [1] "Uncertain DNA"
```

What did we learn?

- First check **critical conditions** (is there an N?)
 - Then check **other criteria** (GC ratio)
-

Exercise 8: list_report(dna_list)

This function prints a report for each DNA sequence in the list.

Example output

1. DNA -> Normal DNA | GC: 0.6
2. DNA -> Normal DNA | GC: 0
3. DNA -> High GC | GC: 1
4. DNA -> Uncertain DNA

```
list_report <- function(dna_list) {  
  
  idx <- 1  
  # idx: counter to show which DNA sequence we are processing  
  
  for (dna in dna_list) {  
  
    check <- dna_check(dna)  
    # first check if the DNA has invalid characters  
  
    if (check != "OK") {  
  
      print(paste(idx, ". DNA ->", check))  
  
    } else {  
  
      status <- dna_quality(dna)  
  
      if (status == "Uncertain DNA") {  
  
        print(paste(idx, ". DNA ->", status))  
  
      } else {  
  
        ratio <- gc_ratio(dna)  
  
        print(paste(idx, ". DNA ->", status, "| GC:", ratio))  
  
      }  
    }  
  }  
}
```

```

    idx <- idx + 1
  }
}

list_report(dna_list)

## [1] "1 . DNA -> High GC | GC: 0.6"
## [1] "2 . DNA -> Normal DNA | GC: 0"
## [1] "3 . DNA -> High GC | GC: 1"
## [1] "4 . DNA -> Uncertain DNA"

```

What did we learn?

- Iterating through a list with `for`
 - Calling functions for each element
 - Combining text using `paste()`
-

Exercise 9: `dna_summary(dna)`

Returns a short summary text for one DNA sequence.

Expected output

```
dna_summary(dna1) = "OK | Normal DNA | GC=0.6"
```

```

dna_summary <- function(dna) {

  check <- dna_check(dna)

  if (check != "OK") {

    return(check)

  } else {

    status <- dna_quality(dna)

    if (status == "Uncertain DNA") {

      return(paste("OK |", status))

    } else {

      ratio <- gc_ratio(dna)

      return(paste("OK |", status, "| GC=", ratio))

    }

  }

}

dna_summary(dna1)

```

```
## [1] "OK | High GC | GC= 0.6"
```

```
dna_summary(dna2)
```

```
## [1] "OK | Normal DNA | GC= 0"
```

```
dna_summary(dna3)
```

```
## [1] "OK | High GC | GC= 1"
```

What did we learn?

- Pipeline logic: **check** → **quality** → (if needed) **GC ratio**
- Creating a single summary output string